

Table of Contents

Documentation technique du projet Claviger

Version documentee : branche main fournie le 7 septembre 2026

Objet : expliquer le fonctionnement reel de la V1 et donner un guide de maintenance pedagogique.

Cette documentation distingue explicitement **l'etat actuel** des recommandations d'architecture. Les fonctionnalites IA, l'interface Web et le deploiement Docker ne sont pas presents dans la branche auditee.

1. Presentation du projet

Claviger est un bot Discord Python dont la V1 automatise l'attribution et la reconciliation de roles utilisateurs a partir de questionnaires.

Deux parcours principaux sont exposes :

- `/membre` : attribue le role membre et synchronise les centres d'interet selectionnes;
- `/noctis` : gere un acces adulte et des categories associees, avec une preference additive pour les variantes generees par IA.

Des commandes administratives permettent egalement de diagnostiquer la base, initialiser/migrer le schema, synchroniser les catalogues, configurer les metadata et analyser la hierarchie des roles.

Le projet est actuellement mono-instance/mono-guild au niveau de l'enregistrement des slash commands, mais la persistance et les policies sont deja scopees par `guild_id`.

2. Vue d'ensemble de l'architecture

Discord

- > Commands / UI
- > Workflow Coordinators
- > Questionnaire / Planner / Executor Services
- > Repositories
- > SQLite

+> ReportService -> Python logging

`-> ReportService -> Forum Discord (optionnel)

L'idée centrale est la **reconciliation** : Claviger ne considère pas un clic comme "ajouter ce rôle". Il compare l'état actuel à l'état souhaité et calcule les différences.

Cette architecture évite plusieurs problèmes :

- reexécuter un questionnaire ne crée pas de doublon;
- des sélections anciennes peuvent être retirées proprement;
- les rôles hors périmètre du workflow sont préservés;
- une UI ouverte depuis plusieurs minutes est révalidée au submit;
- le calcul du plan peut être testé sans Discord.

3. Arborescence commentée

`src/claviger/main.py`

Point d'entrée console. `main()` récupère le token Discord, construit `ClavigerBot`, puis appelle `bot.run(token)`.

Il ne doit pas contenir de logique métier.

`src/claviger/bot.py`

Composition root de l'application. Il :

- crée le client Discord et la `CommandTree`;
- charge l'identifiant du guild principal;
- construit DB, repositories, services, politiques et reporters;
- injecte les dépendances dans les commandes;
- synchronise les slash commands au démarrage.

Le fait que ce fichier connaisse les implémentations concrètes est normal : la composition root est précisément l'endroit où les abstractions rencontrent l'infrastructure.

`src/claviger/commands/`

Adaptateurs Discord des slash commands. Leur responsabilité devrait rester limitée à :

1. vérifier le contexte Discord;
2. vérifier l'autorisation;
3. valider les préconditions de haut niveau;
4. appeler le cas d'usage/coordinator;
5. transformer le résultat en réponse Discord;
6. transmettre les erreurs au mécanisme commun de reporting.

Ils ne doivent pas contenir de SQL ni de logique de calcul de rôles.

[src/claviger/ui/](#)

Modales, checkbox groups et vues Discord.

CatalogSelectionModal fournit une UI generique avec :

- maximum 10 options;
- troncature des labels/descriptions;
- controle actor_id pour empecher un autre utilisateur de soumettre le formulaire.

Les modales membre et Noctis specialisent ensuite cette base.

[src/claviger/services/](#)

Contient la majorite de la logique applicative.

Trois familles apparaissent :

- **planners/workflows purs** : calculs sans I/O (MemberRolePlannerService, CatalogSyncPlanner, AdultAccessWorkflowService);
- **coordinators** : orchestrent plusieurs services;
- **services d'integration Discord** : role discovery, role manager, executors, say.

Cette organisation fonctionne aujourd'hui. Si une seconde interface (Web/agent) reutilise les memes cas d'usage, certains services Discord devront migrer vers integrations/discord ou etre encapsules par des Protocols.

[src/claviger/models/](#)

Dataclasses representant :

- entrees de catalogues;
- questionnaires;
- snapshots Discord;
- plans de synchronisation;
- plans et resultats de mutation de roles.

Les models sont majoritairement immuables (frozen=True), ce qui limite les mutations accidentelles pendant la planification.

[src/claviger/repositories/](#)

Couche de persistance SQLite.

RoleChannelCatalogRepository est le repository generique commun aux centres d'interet et acces adultes. InterestCatalogRepository et AccessCatalogRepository configurent table, colonne de cle et type de model.

[src/claviger/database/](#)

- connection.py : ouverture/fermeture des connexions aiosqlite;

- schema.py : migrations et version du schema;
- status.py : diagnostic non destructif de la DB.

[src/claviger/policies/](#)

Definit les regles effectives d'un serveur : noms/prefixes, salons de workflow et activation des fonctions.

SAFE_DEFAULT_POLICY desactive les fonctions sensibles pour un guild sans configuration.

[src/claviger/reporting/](#)

Infrastructure d'evenements administratifs :

- ReportEvent : donnee structuree;
- Reporter : Protocol d'une destination;
- PythonLoggingReporter : destination logging;
- DiscordForumReporter : destination forum Discord;
- ReportService : fan-out tolerant aux pannes d'un reporter.

4. Cycle des principaux cas d'utilisation

4.1 /membre - ouverture

Utilisateur

- > create_member_command/member
- > DatabaseStatusService.check
- > PolicyResolver.resolve
- > verification du salon de salutations
- > MemberWorkflowCoordinatorService.build_questionnaire
- > verification role-management enabled
- > resolution du role membre
- > MemberInterestQuestionnaireService.build_for_member
- > InterestCatalogRepository.list_for_guild
- > filtre des entrees publiquement pretes
- > restauration des selections via member.roles
- > MemberQuestionnaireModal

4.2 /membre - soumission

Modal

- > actor check
- > defer response
- > MemberWorkflowCoordinatorService.apply_selection

- > resolve role membre
- > reconstruire questionnaire depuis DB + roles courants
- > MemberRolePlannerService.build_plan
- > MemberRoleExecutorService.execute
- > preflight de tous les roles
- > suppressions necessaires
- > ajouts necessaires
- > resultat (IDs ajoutes/retires)
- > reponse utilisateur

Le point pedagogique important est la reconstruction de l'etat avant la planification. La valeur envoyee par la modal n'est jamais consideree comme suffisante a elle seule.

4.3 /noctis - ouverture

Le flux est proche de /membre, avec deux differences metier :

- existence d'un role adulte global;
- association de chaque theme a une variante classique no-ia-* et eventuellement une variante ia-*, plus un role global option-ia.

AdultAccessWorkflowService construit les themes logiques a partir du catalogue technique.

4.4 /noctis - soumission

NoctisRolePlannerService calcule :

- role adulte a ajouter si absent;
- roles classiques pour les themes selectionnes;
- variantes IA si l'option est active;
- role global IA;
- anciens acces geres a retirer.

Les roles non geres par ce workflow restent intacts.

4.5 Synchronisation catalogue

Le catalogue reflète l'etat Discord sans faire du nom Discord l'unique identite persistante.

Etapes :

1. RoleChannelDiscoveryService photographie les roles et salons supportes;
2. CatalogRegistry definit les catalogues selon la policy;
3. CatalogSyncPlanner compare DB et snapshot;

4. il produit creates, refreshes, state_updates, warnings;
5. CatalogSyncCoordinatorService ouvre une transaction;
6. les repositories appliquent les plans sans commit individuel;
7. commit global ou rollback global.

5. Models

Model	Role
RoleChannelCatalogEntry	Representation commune d'un role catalogue et de son mapping salon.
MemberInterest	Alias metier d'une entree catalogue d'interet.
AdultAccess	Alias metier d'une entree catalogue adulte.
MemberInterestQuestionnaire	Options publiques + selections courantes.
AdultAccessTheme	Theme logique avec base et variante IA.
AdultAccessQuestionnaire	Themes + selections + preference IA.
MemberRolePlan / NoctisRolePlan	Mutations desirees, sans I/O.
...ExecutionResult	Mutations effectivement appliquees.
DiscordRoleSnapshot / DiscordChannelSnapshot	Etat observe de Discord au moment d'un catalog sync.
CatalogSyncPlan	Ecritures DB a appliquer apres comparaison.
GuildPolicy	Configuration effective complete.
GuildPolicyOverrides	Champs persistables optionnels.
ReportEvent	Evenement administratif independant de la destination.

RoleChannelCatalogEntry.is_publicly_ready

Une entree publique doit etre :

- enabled;
- configuree (label et description non None);
- encore presente sur Discord;
- techniquement gerable;
- rattachee a un channel present;
- mapping valide;
- conforme a la policy actuelle.

Cette propriete est importante car elle centralise la notion "peut etre propose a un utilisateur".

6. Services

6.1 MemberInterestQuestionnaireService

Entree : guild_id, discord.Member.

Sortie : MemberInterestQuestionnaire.

Il lit le catalogue, ordonne les options et restaure les selections a partir des IDs de roles effectivement portes par le membre.

Etat actuel a simplifier : _is_publicly_available duplique la logique de RoleChannelCatalogEntry.is_publicly_ready.

6.2 MemberRolePlannerService

Service pur.

Entrees : questionnaire courant, cles selectionnees, roles actuels, ID du role membre.

Sortie : MemberRolePlan.

Il refuse une cle inconnue et ne retire que les roles faisant partie des interets exposes par le questionnaire.

6.3 MemberRoleExecutorService

Depend de RoleManager et de Discord.

Avant mutation, il :

- verifie que le bot existe dans le guild;
- resout tous les role IDs;
- refuse un role absent;
- refuse un role managed ou au-dessus du bot.

Puis il retire et ajoute les roles.

Limite actuelle : plusieurs appels Discord ne constituent pas une transaction. Un echec intermediaire doit etre reporte comme execution partielle.

6.4 MemberWorkflowCoordinatorService

Orchestre questionnaire, planner et executor. Il est le point d'entree logique du workflow membre.

6.5 AdultAccessWorkflowService

Transforme des entrees techniques no-ia-*/ia-* en themes logiques. Il valide les cles selectionnees et resout les role IDs desires.

Il s'agit d'une logique metier Noctis, pas d'un utilitaire generique.

6.6 NoctisRolePlannerService

Planifie l'ajout du role adulte, du role IA et des variantes de themes. Il calcule egalement les anciennes permissions a retirer.

6.7 NoctisRoleExecutorService

Equivalent technique du member executor. Cette duplication doit etre fusionnee apres la V1 dans un executor generique de mutations de roles.

6.8 CatalogSyncPlanner

Service pur et relativement riche. Il :

- detecte les nouveaux roles correspondant au prefixe;
- rafraichit les donnees Discord modifiees;
- conserve les identites historiques lorsqu'un role disparaît;
- detecte les mappings salon absents ou ambigus;
- met a jour les flags de validite.

6.9 CatalogSyncCoordinatorService

Prend un snapshot unique puis applique tous les catalogues dans une transaction SQLite commune.

6.10 AuthorizationService

Definit des Capability plutot que des checks ad hoc.

Actuellement :

- owner : toujours autorise;
- role de confiance au-dessus de Claviger : SAY;
- SAY_PLAIN : non accorde par default, donc owner uniquement.

La hierarchie au-dessus de Claviger correspond volontairement, dans la regle metier actuelle, a un perimetre de confiance. AuthorizationService traduit ensuite cette confiance en capabilities effectivement exposees par le code (SAY aujourd'hui, tandis que SAY_PLAIN reste reserve a l'owner). Cette convention ne doit pas etre extrapolee automatiquement aux futurs outils IA : les actions sensibles ou destructives devront disposer de capabilities explicites et pourront exiger une confirmation humaine.

6.11 ReportService

Fan-out vers plusieurs reporters. Une destination en erreur n'empêche pas les suivantes de recevoir l'événement.

C'est une bonne place pour conserver le contrat d'observabilité sans faire dépendre les cas d'usage de Discord Forum, Sentry ou d'un format de logs particulier.

7. Acces aux donnees

7.1 Base SQLite

Chemin par défaut :

data/claviger.db

Le chemin est configurable via DATABASE_PATH.

7.2 Version du schema

Version auditee : CURRENT_SCHEMA_VERSION = 4.

Migrations principales :

- v2 : guild_settings;
- v3 : guild_member_interests;
- v4 : guild_adult_accesses.

La version est stockée dans PRAGMA user_version.

7.3 Connexions

DatabaseConnection.connect() est un async context manager. Chaque opération ouvre une connexion et la ferme dans finally.

PRAGMA foreign_keys = ON est exécuté sur chaque connexion.

7.4 Transactions

Les migrations utilisent BEGIN IMMEDIATE et rollback en cas d'erreur.

Le catalog sync ouvre également BEGIN IMMEDIATE et passe la même connexion aux deux repositories, qui n'effectuent pas leur propre commit dans apply_sync_plan.

C'est nécessaire pour garantir qu'un catalogue d'intérêts ne soit pas valide si le catalogue adulte échoue dans la même synchronisation.

7.5 Identites

Pour les catalogues, la cle primaire est (guild_id, role_id). Le nom du role et la cle metier sont refreshables.

Architecture recommandee : ajouter une verification d'unicite de la cle logique par guild pour eviter deux roles avec le meme interest_key/access_key.

8. Configuration

8.1 Variables disponibles

DISCORD_TOKEN=<DISCORD_TOKEN>

DISCORD_GUILD_ID=<DISCORD_GUILD_ID>

DATABASE_PATH=data/claviger.db

ADMIN_REPORT_FORUM_ID=<ADMIN_REPORT_FORUM_ID_OPTIONNEL>

Ne jamais versionner le vrai .env. Le .gitignore audite ignore .env, les bases .db, WAL et SHM.

8.2 Comportement actuel

- get_discord_token() charge .env et exige le token;
- get_discord_guild_id() charge .env, exige l'ID puis convertit en int;
- DATABASE_PATH a un default;
- ADMIN_REPORT_FORUM_ID est optionnel.

Architecture recommandee

Charger une seule fois un objet de settings au demarrage, avec validation des entiers et chemins. Cela permet :

- fail-fast avant la connexion Discord;
- message d'erreur de configuration uniforme;
- tests plus simples;
- pas de dependance implicite au fait qu'une autre fonction ait deja appele load_dotenv().

9. Gestion des erreurs

Etat actuel

Le projet possede deja plusieurs exceptions metier utiles :

- DatabaseUnavailableError, DatabaseMissingError;
- MemberWorkflowError et sous-types;

- NoctisWorkflowError et sous-types;
- erreurs d'execution de roles;
- erreurs de bootstrap et catalogue.

Mais les boundaries les capturent souvent avec except Exception et ne profitent donc pas de cette typologie.

Plusieurs UI renvoient un message propre sans journaliser la cause.

Architecture recommandee

Erreur basse couche

- > exception contextualisee
- > coordinator/service
- > boundary Discord commun
- > log local avec traceback + contexte
- > ReportEvent sanitize si necessaire
- > reponse utilisateur stable

Les exceptions attendues (role manquant, workflow disabled, validation) ne doivent pas toutes etre loggees comme ERROR. Certaines sont WARNING ou simplement une reponse fonctionnelle.

Les erreurs inattendues doivent conserver exc_info.

10. Logs et reporting

Etat actuel

Claviger sait produire des ReportEvent INFO/WARNING/ERROR/CRITICAL et les envoyer :

- au logging Python;
- a un forum Discord optionnel.

ReportService isole la panne d'un reporter.

Limites :

- pas de configuration logging centrale;
- print au demarrage;
- pas de DEBUG applicatif defini;
- pas de format JSON;
- champs event flatten en texte;
- pas de correlation/request ID;
- stack traces perdues pour les erreurs applicatives;
- integration incomplete des workflows publics.

Architecture recommandee

Garder ReportEvent pour les evenements administratifs semantiques et utiliser le logger Python pour les traces techniques.

Ne pas transformer chaque ligne DEBUG en ReportEvent.

11. Tests

11.1 Etat actuel

252 fonctions de test ont ete retrouvees statiquement.

Repartition notable :

- services;
- repositories;
- database;
- commands;
- UI;
- reporting;
- policies;
- composition du bot.

tests/test_claviger/ est un dossier historique issu du decoupage d'un ancien module Claviger trop volumineux. Il ne doit pas etre lu comme une architecture concurrente intentionnelle.

11.2 Lancer les tests

En environnement de developpement configure :

```
python -m pytest
```

Cette commande demande a Python d'executer Pytest comme module. pyproject.toml fixe testpaths = ["tests"], donc Pytest decouvre la suite dans ce dossier.

Pour un test cible :

```
python -m pytest tests/services/test_member_role_executor_service.py
```

Cela reduit la boucle de feedback pendant un travail sur l'executor.

11.3 Lint

Le projet declare Ruff en dependance de developpement :

```
ruff check .
```

Ruff verifie actuellement les erreurs Python principales, imports et quelques regles E/F/I. Apres stabilisation, des regles supplementaires peuvent etre activees progressivement, sans faire d'un linter une fin en soi.

11.4 Organisation recommandee

Voir le rapport d'audit pour la cible unit/, integration/, component/discord/, smoke/.

12. Ajouter une nouvelle fonctionnalite

Cas A - Nouvelle logique de role utilisant Discord

Exemple conceptuel : nouveau questionnaire communautaire.

Model de questionnaire/plan

- > Planner pur
- > Executor generique de roles
- > Coordinator metier
- > Command / Modal Discord
- > Tests unitaires planner
- > Tests executor
- > Tests command/UI

Le planner ne doit pas recevoir une discord.Interaction.

Cas B - Nouvelle commande de lecture admin

Command

- > authorization/owner check
- > service de lecture
- > repository ou integration Discord
- > formatage de reponse
- > ReportEvent seulement si l'evenement est administrativement utile

Cas C - Nouvelle table/metadonnee

1. Ajouter une migration de schema versionnee.
2. Ajouter/adapter model.
3. Ajouter repository.
4. Tester migration depuis la version precedente.
5. Tester rollback et isolation par guild.
6. Exposer via service/coordinator.
7. Ajouter UI en dernier.

Cas D - Futur outil IA

Ne pas laisser le LLM appeler directement un repository ou RoleManager.

Cible :

LLM/Agent

- > Tool schema
- > authorization(capability, actor, guild)
- > validation
- > application service
- > audit event
- > integration Discord/DB
- > result structure

Une action destructive doit pouvoir exiger confirmation ou capability plus forte.

13. Guide de developpement

13.1 Prerequis

- Python 3.12 ou version compatible definie par le projet;
- acces a un bot Discord de developpement;
- variables d'environnement dediees;
- environnement virtuel recommande.

13.2 Creer l'environnement virtuel

```
python -m venv .venv
```

Cette commande cree un Python isole dans .venv. Elle evite de melanger les dependances Claviger avec celles d'autres projets.

Activation sous PowerShell :

```
.\.venv\Scripts\Activate.ps1
```

Sous Linux :

```
source .venv/bin/activate
```

13.3 Installer le projet en mode developpement

```
python -m pip install -e "[dev]"
```

-e installe le package en editable : modifier src/claviger modifie directement le code utilise sans reinstallation. [dev] ajoute Pytest, pytest-asyncio et Ruff.

13.4 Configurer l'environnement

Copier .env.example vers .env, puis renseigner uniquement des secrets/IDs de developpement.

Ne jamais utiliser le token de production pour les tests locaux.

13.5 Initialiser la base

La conception actuelle impose une initialisation explicite via les commandes administratives Claviger. Le status permet de verifier si la DB est absente, non initialisee, a migrer, trop recente ou prete.

Cette approche est volontaire : une simple lecture ne doit pas creer silencieusement une nouvelle DB.

13.6 Lancer Claviger

Apres installation du package, l'entry point declare est :

claviger

Equivalent conceptuel a l'appel de claviger.main:main.

13.7 Verification avant commit

Ordre recommande :

ruff check .

python -m pytest

Puis test fonctionnel Discord si une modification touche commands/UI/permissions.

14. Deploiement

Etat actuel

Aucun Dockerfile, compose file, service systemd ou pipeline de deploiement n'est present dans la branche auditee. Le deploiement Docker sur mini-PC est prevu ulterieurement et doit etre traite dans la session dediee.

Il serait incorrect de documenter un deploiement actuel qui n'existe pas encore.

Exigences a conserver pour le futur Docker

Sans fournir ici l'implementation :

- image Python reproductible;
- utilisateur non-root si possible;
- token injecte par variable/secrets, jamais bake dans l'image;
- volume persistant pour data/claviger.db;
- sauvegarde/restauration testee de SQLite;
- logs sur stdout/stderr;
- limite de taille/retenue des logs Docker;
- politique de restart;

- execution des migrations explicitement controlee;
- separation des variables dev/prod.

15. Glossaire

Catalog - Liste persistante de roles Discord geres par un workflow avec metadata utilisateur.

Catalog key - Cle metier derivee du nom du role, distincte de l'ID Discord.

Coordinator - Service qui orchestre plusieurs composants pour realiser un cas d'usage.

Executor - Composant qui applique un plan sur un systeme externe.

Planner - Composant pur calculant les changements necessaires sans I/O.

Policy - Configuration effective des conventions et fonctions d'un guild.

Preflight - Validation de toutes les operations prevues avant la premiere mutation.

Reconciliation - Calcul de la difference entre etat actuel et etat desire.

Repository - Couche d'accès aux donnees persistantes.

Reporter - Destination recevant un ReportEvent.

Snapshot - Representation immutable de l'etat Discord observe a un instant.

Stale UI - Interface ouverte sur un ancien etat devenu obsolete avant sa soumission.

Guild - Serveur Discord.

Capability - Droit applicatif explicite (SAY, etc.) independant d'une commande particuliere.

Conclusion

Pour travailler proprement sur Claviger, la regle la plus importante est de conserver le sens du decoupage existant : **observer/reconstruire -> planifier -> valider -> executer -> reporter**.

Les prochaines evolutions doivent reutiliser cette logique plutot que remettre les appels Discord, la persistence et les regles metier directement dans les commands. La priorite immediate est de rendre les erreurs et l'etat d'exploitation aussi explicites que les plans de roles le sont deja.